

import libraries

```
# Numerical operations
import numpy as np

# Data handling
import pandas as pd

# Text preprocessing
import re

# PyTorch libraries
import torch
import torch.nn as nn
import torch.optim as optim

# Dataset splitting & evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Optional visualization
import matplotlib.pyplot as plt
```

Load and explore dataset

```
df = pd.read_csv('spam.csv', encoding='latin-1')[['v1', 'v2']]
df.columns = ['label', 'text']

print(df.head())
print(df['label'].value_counts())
```

```
   label      text
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...
label
ham      4825
spam      747
Name: count, dtype: int64
```

Text preprocessing

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z', ' ', text)
    return text

df['text'] = df['text'].apply(clean_text)

# tokenisation
from collections import Counter

words = " ".join(df['text']).split()
vocab = Counter(words)

#padding

MAX_LEN = 50

def encode(text, word2idx):
    tokens = text.split()
    seq = [word2idx.get(word, 0) for word in tokens]
    if len(seq) < MAX_LEN:
        seq += [0] * (MAX_LEN - len(seq))
    else:
        seq = seq[:MAX_LEN]
    return seq
```

vocabulary and encoding

```
word2idx = {word: i+1 for i, (word, _) in enumerate(vocab.items())}

X = np.array([encode(text, word2idx) for text in df['text']])
```

```
y = np.array([1 if label == 'spam' else 0 for label in df['label']])
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Build 1d CNN with dropout

```
class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super(TextCNN, self).__init__()

        self.embedding = nn.Embedding(vocab_size, embed_dim)

        self.conv1 = nn.Conv1d(embed_dim, 100, kernel_size=3)
        self.pool = nn.MaxPool1d(2)

        self.dropout = nn.Dropout(0.5)

        self.fc = nn.Linear(100 * 24, 1) # depends on sequence size

    def forward(self, x):
        x = self.embedding(x)          # (batch, seq, embed)
        x = x.permute(0, 2, 1)         # (batch, embed, seq)

        x = self.conv1(x)
        x = torch.relu(x)
        x = self.pool(x)

        x = x.view(x.size(0), -1)

        x = self.dropout(x)

        x = self.fc(x)
        return torch.sigmoid(x)
```

Model Training

```
model = TextCNN(len(word2idx)+1, 50)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

losses = []

for epoch in range(5):
    model.train()

    inputs = torch.tensor(X_train, dtype=torch.long)
    labels = torch.tensor(y_train, dtype=torch.float32)

    optimizer.zero_grad()

    outputs = model(inputs).squeeze()
    loss = criterion(outputs, labels)

    loss.backward()
    optimizer.step()

    losses.append(loss.item())

    print(f"Epoch {epoch+1}, Loss: {loss.item()}")
```

```
Epoch 1, Loss: 0.6726495623588562
Epoch 2, Loss: 0.47515159845352173
Epoch 3, Loss: 0.403109610080719
Epoch 4, Loss: 0.4018608629703522
Epoch 5, Loss: 0.41593971848487854
```

Model Evaluation

```
model.eval()

inputs = torch.tensor(X_test, dtype=torch.long)
preds = model(inputs).detach().numpy()
```

```
preds = (preds > 0.5).astype(int)

print("Accuracy:", accuracy_score(y_test, preds))
print("Precision:", precision_score(y_test, preds))
print("Recall:", recall_score(y_test, preds))
print("F1 Score:", f1_score(y_test, preds))
print("Confusion Matrix:\n", confusion_matrix(y_test, preds))
```

Accuracy: 0.8654708520179372

Precision: 0.0

Recall: 0.0

F1 Score: 0.0

Confusion Matrix:

[[965 0]

[150 0]]

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-de
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))